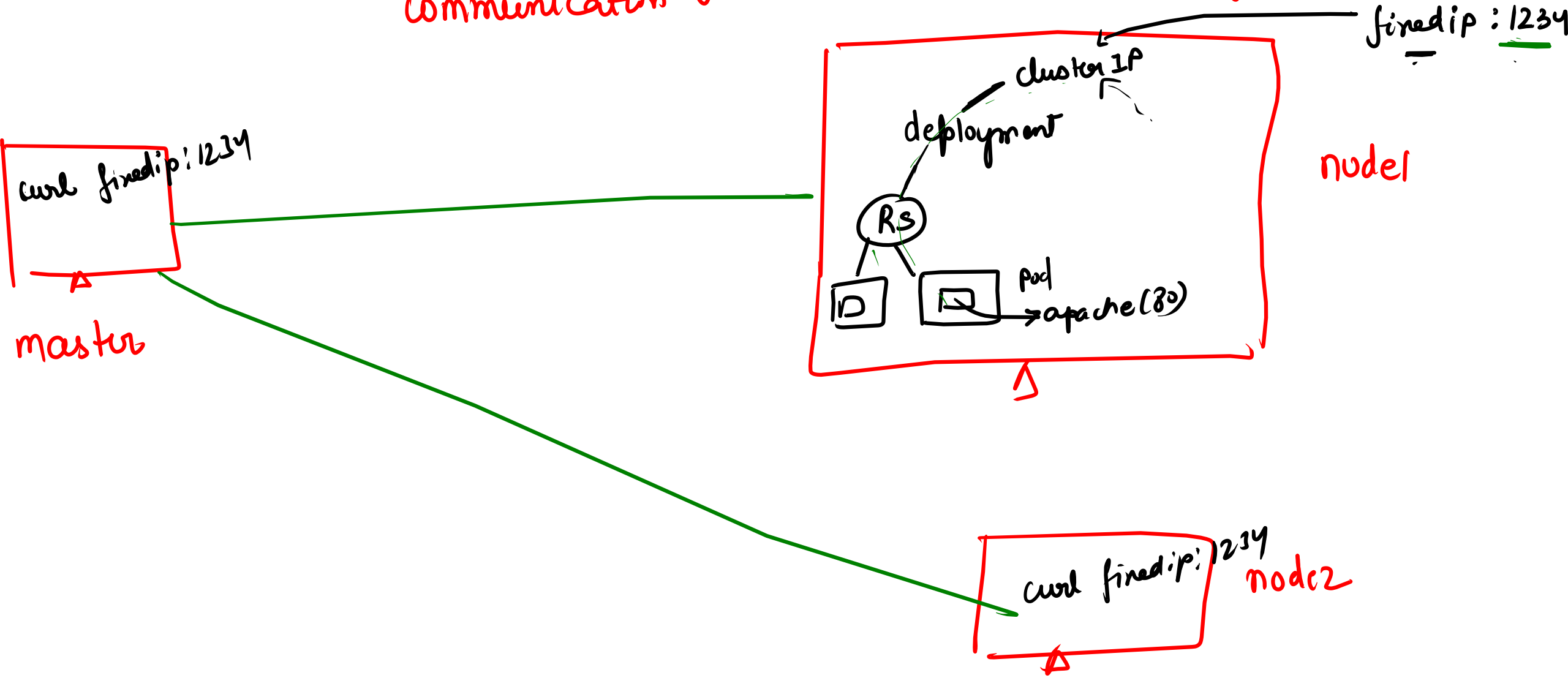
Cluster IP

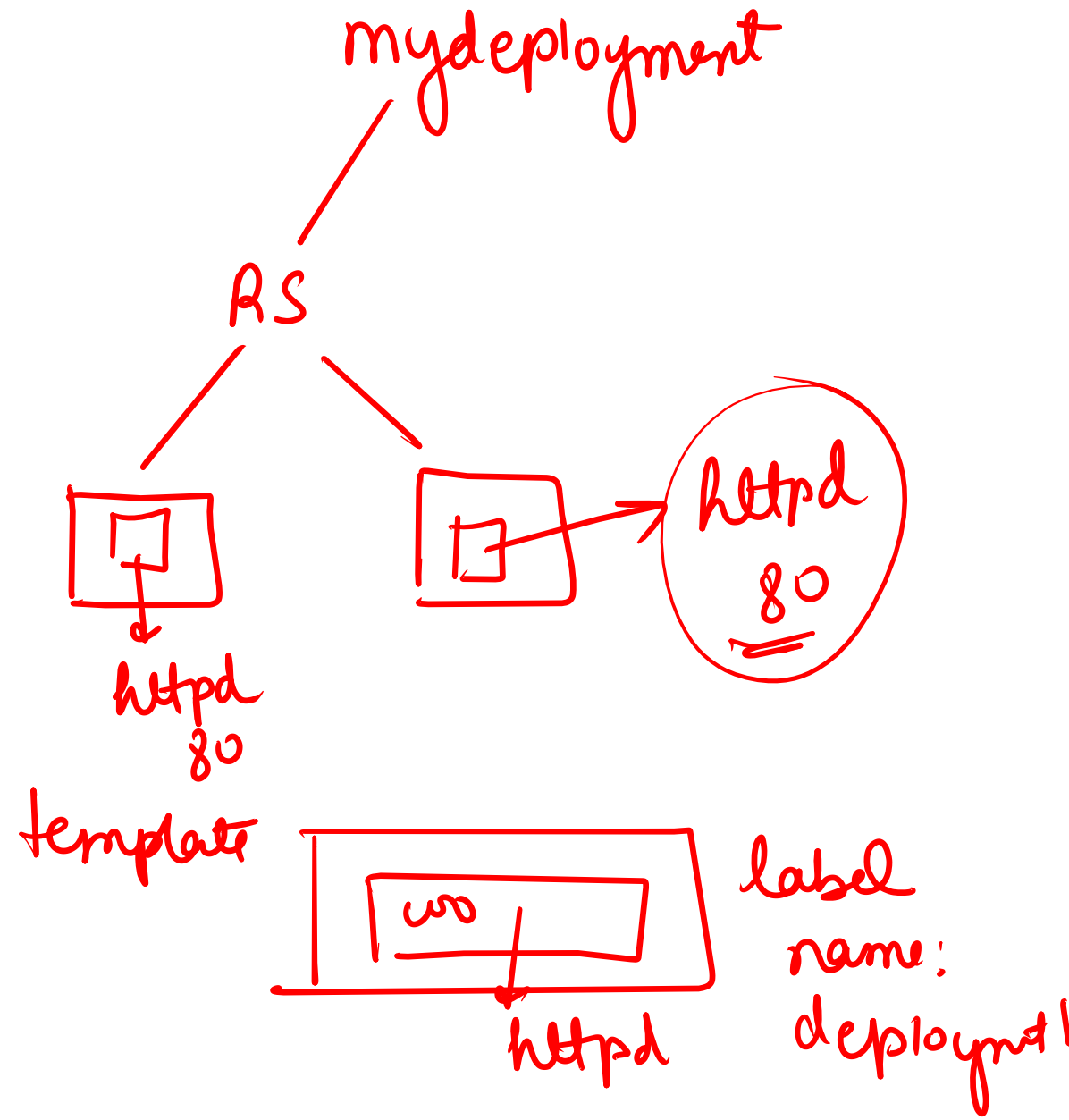
→ Accsible only within the cluster. useful for Internal communication



vi deployhttpd1.yml

```yaml
kind: Deployment
apiVersion: apps/v1
metadata:
  name: mydeployment
spec:
  replicas: 2
  selector:
    matchLabels:
      name: deployment1
  template:
    metadata:
      name: testp1
      labels:
        name: deployment1
    spec:
      containers:
        - name: c00
          image: httpd
          ports:
            - containerPort: 80
```

The Replica pods will be created from the template label name: deployment1



```
root@ip-172-31-6-141:/home/ubuntu# kubectl apply -f deployhttpd1.yml
deployment.apps/mydeployment created
root@ip-172-31-6-141:/home/ubuntu#
```

```
root@ip-172-31-6-141:/home/ubuntu# kubectl get deploy
NAME           READY   UP-TO-DATE   AVAILABLE   AGE
mydeployment   2/2     2            2           21s
root@ip-172-31-6-141:/home/ubuntu# kubectl get rs
NAME                     DESIRED   CURRENT   READY   AGE
mydeployment-58cbb5545d  2         2         2       32s
root@ip-172-31-6-141:/home/ubuntu# kubectl get pods -o wide
NAME                              READY   STATUS    RESTARTS   AGE   IP               NODE                 
mydeployment-58cbb5545d-fd7gc     1/1     Running   0          39s   192.168.122.89   ip-172-31-15-78      
mydeployment-58cbb5545d-jxc9x     1/1     Running   0          39s   192.168.9.153    ip-172-31-2-109      
root@ip-172-31-6-141:/home/ubuntu#
```

get Replicaset

ip address of pods

curl ipaddress of pod

```
root@ip-172-31-6-141:/home/ubuntu# curl 192.168.122.89
<html><body><h1>It works!</h1></body></html>
root@ip-172-31-6-141:/home/ubuntu# curl 192.168.9.153
<html><body><h1>It works!</h1></body></html>
root@ip-172-31-6-141:/home/ubuntu#
```

```
root@ip-172-31-6-141:/home/ubuntu# kubectl delete pods --all
pod "mydeployment-58cbb5545d-fd7gc" deleted
pod "mydeployment-58cbb5545d-jxc9x" deleted
root@ip-172-31-6-141:/home/ubuntu# curl 192.168.9.153
```

It will not work anymore :(

end point is changed

Now we will create a service - Cluster ip which will provide the fixed ip address

vi service.yml

```yaml
kind: Service
apiVersion: v1
metadata:
  name: demoservices
spec:
  ports:
    - port: 8080
      targetPort: 80
  selector:
    name: deployment1
  type: ClusterIP
```

we are creating a service

for the kind as service the api version is v1

The name of the service is demo services.

fixed ip will be mapped on port 8080

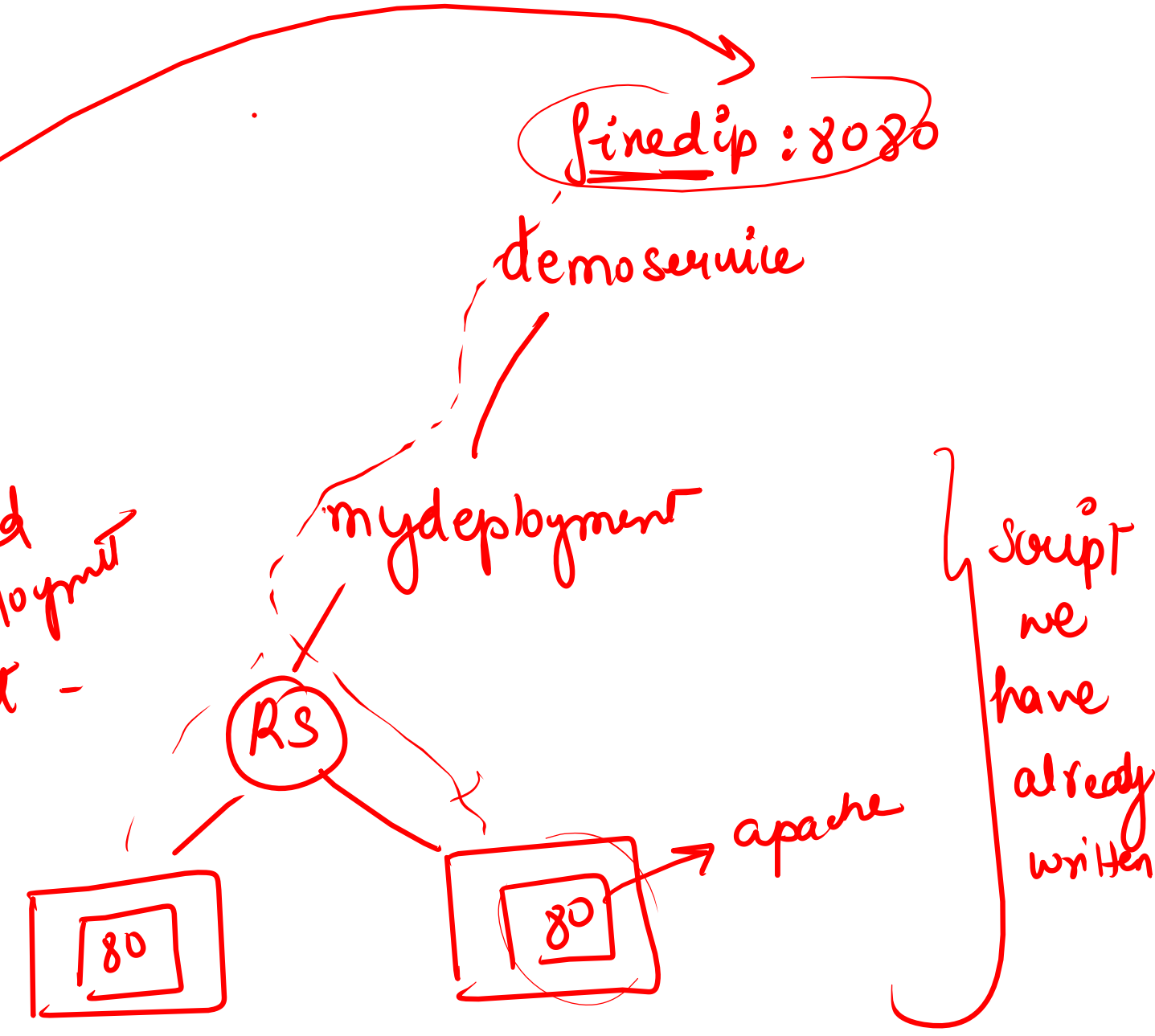
The pod port on which app is running is port 80

label of the deployment

we are creating a cluster ip

```yaml
kind: Service
apiVersion: v1
metadata:
  name: demoservices
spec:
  ports:
    - port: 8080
      targetPort: 80
  selector:
    name: deployment1
  type: ClusterIP
```

label we used in deployment script -



```
root@ip-172-31-6-141:/home/ubuntu# kubectl apply -f service.yml
service/demoservices created
root@ip-172-31-6-141:/home/ubuntu# kubectl get service
NAME            TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)    AGE
demoservices    ClusterIP   10.100.254.231   <none>        8080/TCP   10s
kubernetes      ClusterIP   10.96.0.1        <none>        443/TCP    2d
root@ip-172-31-6-141:/home/ubuntu#
```

get list services

fixed ip (automatically assigned)

port which we mapped

```
root@ip-172-31-6-141:/home/ubuntu# kubectl get service
NAME            TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)    AGE
demoservices    ClusterIP   10.100.254.231   <none>        8080/TCP   10s
kubernetes      ClusterIP   10.96.0.1        <none>        443/TCP    2d
root@ip-172-31-6-141:/home/ubuntu# curl 10.100.254.231:8080
<html><body><h1>It works!</h1></body></html>
root@ip-172-31-6-141:/home/ubuntu#
```

lets delete all the pods and see if the same ip still works or not

```
root@ip-172-31-6-141:/home/ubuntu# kubectl delete pods --all
pod "mydeployment-58cbb5545d-5lmq2" deleted
pod "mydeployment-58cbb5545d-wvtmn" deleted
root@ip-172-31-6-141:/home/ubuntu# curl 10.100.254.231:8080
<html><body><h1>It works!</h1></body></html>
root@ip-172-31-6-141:/home/ubuntu#
```

deleted the pods

final ip